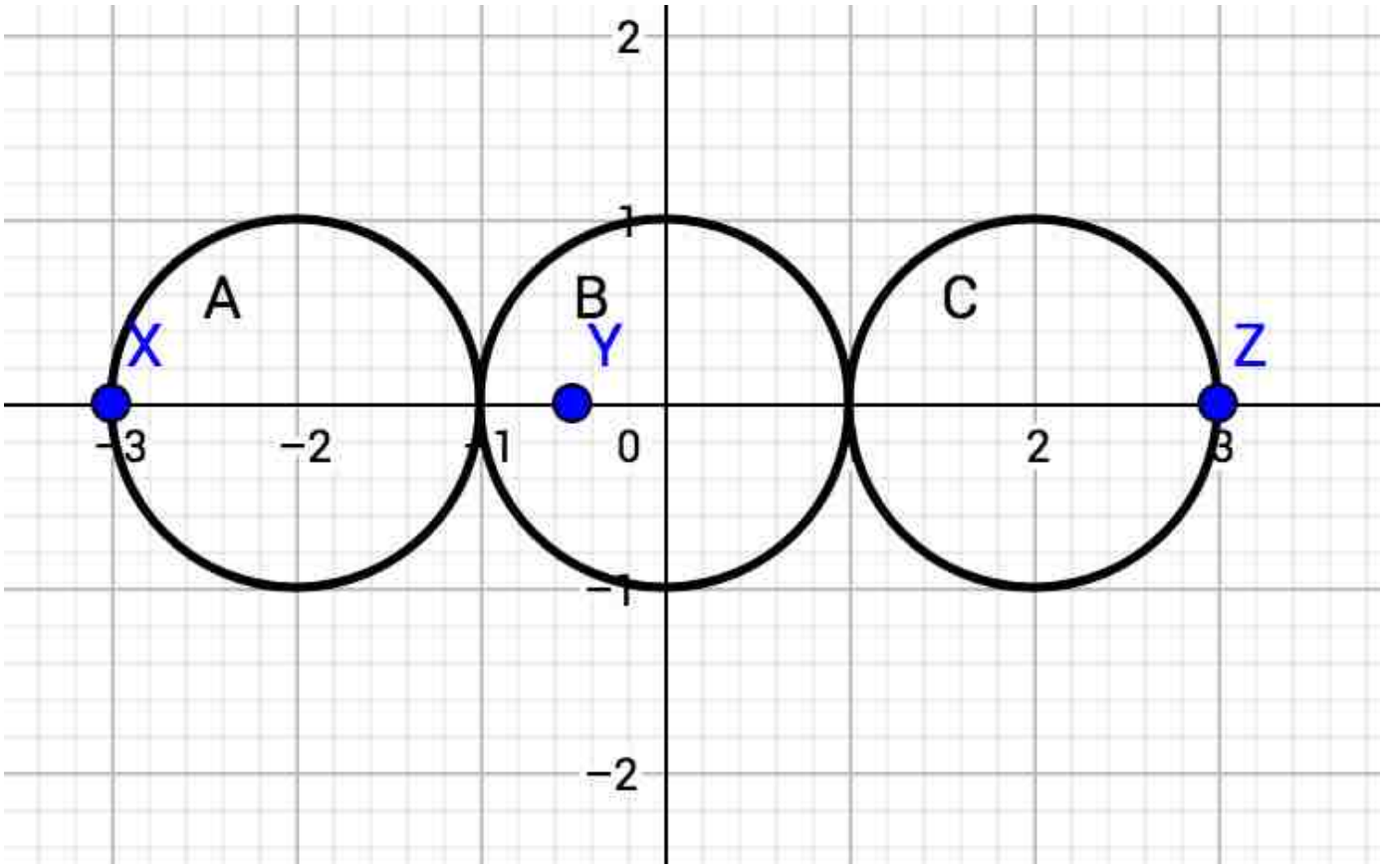
Kick Start 2017 - Round E

Analysis: Blackhole

Black Hole: Analysis

Small Dataset

In the Small dataset, all three black holes are along the x-axis, so the solution is simple: the three spheres should be placed in a row (touching only at single points) to cover the distance spanned by the leftmost and rightmost of the black holes. So the answer is just the distance between leftmost and rightmost black holes, divided by 6.

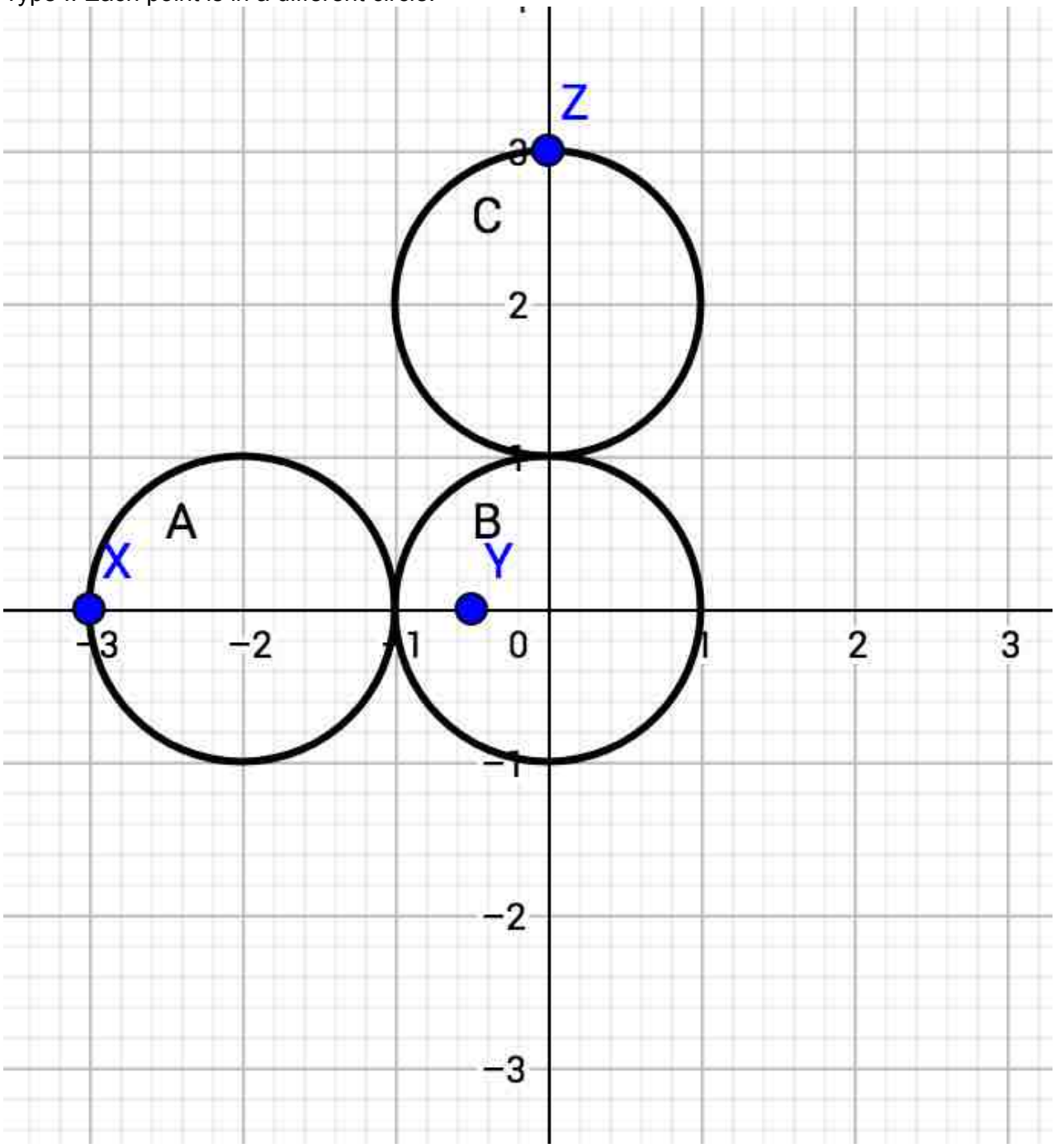


Large Dataset

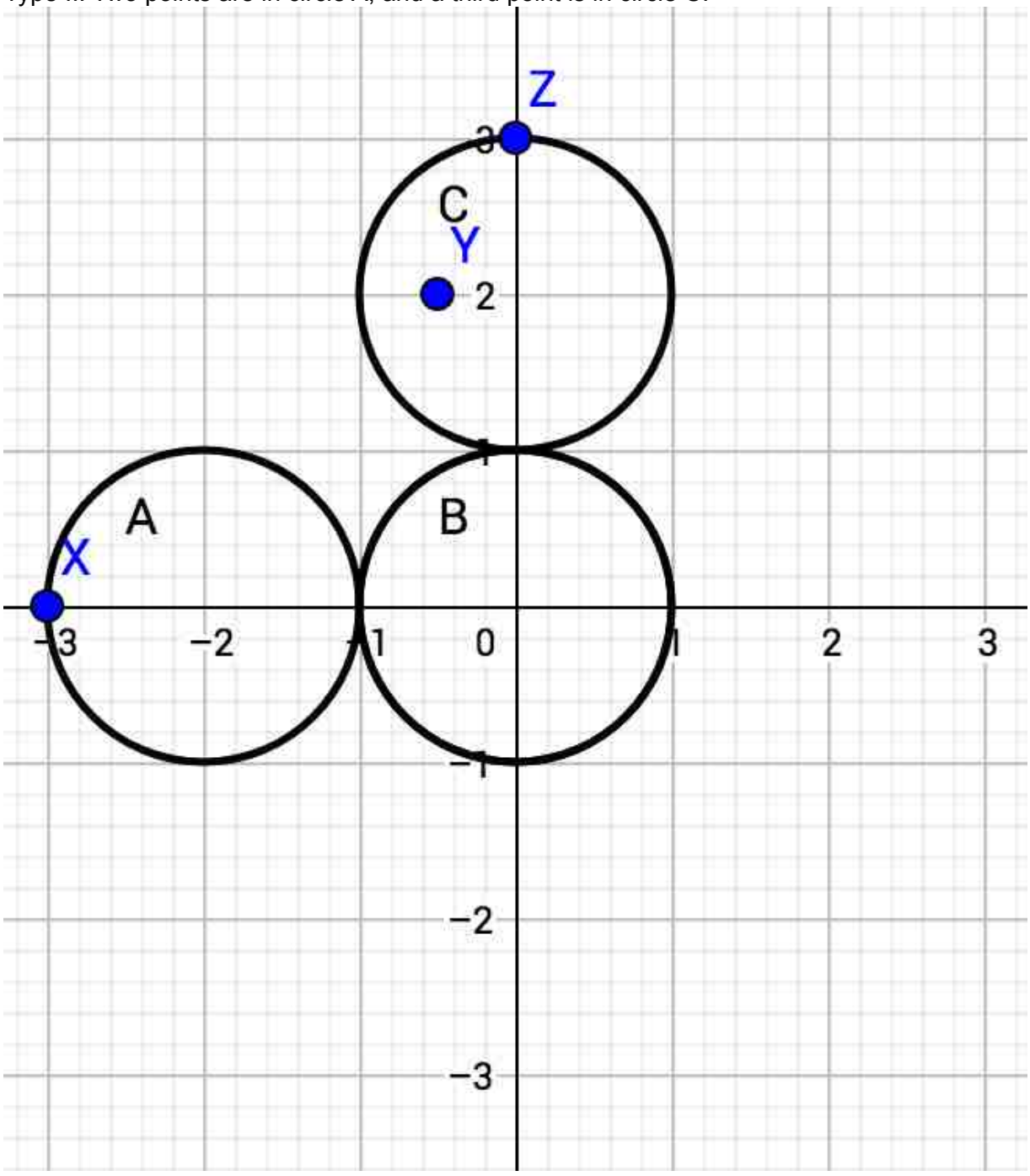
To solve the Large dataset, it is helpful to notice that although this is a 3-D problem, it only has three points (black holes), and there is at least one plane that contains all three of the points. It is therefore possible to simplify the problem into a 2-D problem: how can we use three connected circles of equal radius to cover three points on a plane?

Suppose that we have Circles A, B, and C, with circle A connected to circle B, circle B connected to circle C, and circles A and C possibly (but not necessarily) connected. Then these are the only two potentially optimal ways to cover the points:

Type I: Each point is in a different circle.



Type II: Two points are in circle A, and a third point is in circle C.



If we call the points X, Y, and Z, then there are three Type I possibilities and three Type II possibilities. For example, one possibility for Type I is to have Point X in Circle A, Point Y in Circle B, and Point Z in Circle C. One possibility for Type II is to have Points Y and Z in Circle A, and Point X in Circle C.

Any other arrangement cannot be optimal. For instance, if all three points are in one circle, we are wasting the other two circles. If two points are in one circle and one point is in another circle, and the third circle does not connect the first two, then we could make the circles smaller and use the third to connect the first two.

To find the minimum valid radius for a possible configuration, we can use binary search. But we still need a way to determine whether a radius is valid.

For Type I possibilities, how can we determine whether a radius R is valid? Let K be the center of circle B. Then:

1. The distance between K and the point in circle B must be no greater than R, or else the point would not be in B.

2. The distance between K and the point in circle A must be no greater than 3R, or else circles A and B would not be connected.
3. The distance between K and the point in circle C must be no greater than 3R, or else circles B and C would not be connected.

It is equivalent to check that point K is in all three of these circles:

1. A circle of radius R centered at the point in circle B
2. A circle of radius 3R centered at the point in circle A
3. A circle of radius 3R centered at the point in circle C

So, if those three circles have any common intersection, then we know that radius R is valid, without having to choose a specific point K.

We can use a similar strategy for Type II possibilities. Let K be the center of circle A. Then:

1. The distance between K and the point in circle C must be no greater than 5R, or else circles A and C would not be connected.
2. The distance between K and the first point in circle A must be no greater than R, or else the point would not be in circle A.
3. The same is true of the second point in circle A.

and so we only need to look for a common intersection of these three circles:

1. A circle of radius 5R centered at the point in circle C
2. A circle of radius R centered at the first point in circle A
3. A circle of radius R centered at the second point in circle A

To determine whether the three circles have at least one point in common, we can start by finding the intersections of the perimeters of each pair of circles. If two circles' perimeters intersect at...

- ...two points, we can check whether either of those points is in the third circle.
- ...one point, we can check whether that point is in the third circle.
- ...infinitely many points (that is, the two circles are the same), it suffices to check whether either of those circles intersects the third circle.
- ...zero points, then either one circle is completely within the other, or the circles are disjoint. If the circles are disjoint, there can be no common point. Otherwise, we can disregard the outermost circle and check whether the innermost circle intersects the third circle.

If none of these checks succeed, then the three circles have no point in common. Otherwise, they have at least one.

Once we have found the minimum radius for each cover method, the minimum of those is our final answer.

Kick Start 2017 - Round E

Analysis: Copy & Paste

Copy & Paste: Analysis

Small dataset

Let N be the length of the target string **S**. We can observe that:

- Copying and pasting a single character is always strictly worse than just typing another instance of that character, which takes only one operation.
- Copying and pasting two or more characters can be useful, but it turns out not to be for N ≤ 5. Suppose that we wanted to use copying and pasting to create a string of length 5. Then the only possible copy/paste operation that uses two or more characters is to start with a string of length 3 and copy/paste two of its characters. But then we could have just typed the two characters instead, using the same number of operations.
- For N = 6, there are some patterns for which the optimal strategy requires copying and pasting. With some brute-force experimentation or case-based analysis, you can find that they are of all of the of the form 111111, 121212, or 123123 (in which each number consistently stands for a particular letter)..

It is also possible to use a brute-force breadth/depth-first search to solve the Small.

Large dataset

For the Large dataset, we can use dynamic programming. We will keep track of the minimum number of operations used to reach a state with the following parameters, as the target string is being built from left to right:

- Current length - the length of the prefix of the target string that we have built so far
- Clipboard content - the content in the clipboard

There can be at most $O(N^2)$ substrings, so there are $O(N^3)$ states in total. For each state, we can choose to type a character, paste the contents of the clipboard, or copy something into the clipboard. Since there are $O(N^2)$ substrings that we could choose to copy, there are $O(N^2)$ possible moves from each of the $O(N^3)$ possible states, and so the overall time complexity is $O(N^5)$. This approach is fast enough to pass all the test cases within the time limit.

Notice that the copy operation is needed only when the copied string needs to be pasted as the next operation. It reduces the number of candidate strings that need to be copied in each move to O(N) because the number of substring starting with next character is O(N). Thus, this approach leads to an $O(N^4)$ solution.

There is even an $O(N^3)$ approach. Can you find it?

Kick Start 2017 - Round E

Analysis: Trapezoid Counting

Trapezoid Counting: Analysis

Small dataset

There are at most 50 sticks, which means we can enumerate all different sets of four sticks directly.

For a set of sticks to form the four sides of an isosceles trapezoid, it must meet all of the following conditions:

- There must be a pair of sticks with equal length. Call this length C.
- The remaining two sticks must have unequal lengths. Call the shorter of those lengths A, and the longer one B.
- The four sticks must be able to actually connect to form an isosceles trapezoid, so we need to obey an isosceles trapezoid equivalent of the triangle inequality: $B < A + 2 \times C$.

Then we just need to count how many different sets of four sticks meet these conditions.

Large dataset

First, we can create a Length array with all the unique length values, and a Count array with the counts of each of those values.
For example, for sticks with lengths [3, 4, 1, 4, 2, 6, 3, 1, 3], we would get the Length array [1, 2, 3, 4, 6] and the Count array [2, 1, 3, 2, 1].

Consider the inequality above: $B < A + 2 \times C$.
There are two possible situations in which valid isosceles trapezoids can be formed:

- A is equal to C or B is equal to C.
- A, B, and C are all different values. (Recall that we cannot have A equal B, or else the shape would not meet the problem's definition of an isosceles trapezoid.)

For the first situation, we consider all possible values C = Length[i] such that Count[i] ≥ 3.
For each of these, we have to count how many sticks have length less than 3 × C but not equal to C.
We can do this quickly by using a prefix sum of the Count array (which we only need to create once). Then we add that number, multiplied by (Count[i] choose 3), to our answer.

For the second situation, we consider all possible values C = Length[i] such that Count[i] ≥ 2.
For each of these, we consider all A = Length[j] (with $i \neq j$).
For each of those (C, A) pairs, we need to count how many sticks have length B such that $B \neq C$, $A < B$, and $B < A + 2 \times C$.
The number of valid B is the prefix sum of Count up to A + 2 × C, minus the prefix sum of Count up to A, possibly minus the Count of C if it is in that range.
Then we add that number, multiplied by (Count[i] choose 2), to our answer.

This method avoids double-counting any sets, and it runs in $O(\mathbf{N}^2)$ time, which is easily fast enough for $\mathbf{N} \leq 5000$.